

GA Sudoku Solver (PyGAD)

Minimal Documentation

Project Documentation

October 17, 2025

Repository: https://github.com/ArmanBjr/Artificial_Intelligence-Course

Document version: October 17, 2025

Abstract

This document provides a concise, contributor-facing overview of the Sudoku solver implemented with a Genetic Algorithm (PyGAD). It focuses on the important public functions and how they fit together. The style and package set are kept lightweight for GitHub repositories.

Contents

1	Overall Overview	1
2	Quick Start	2
3	Repository Structure	2
4	Key Components	2
4.1	Class <code>SudokuSolver</code>	2
4.2	Utilities	2
5	Example Snippet	2

1 Overall Overview

This project solves Sudoku puzzles using a Genetic Algorithm (GA) built on `pygad`. Only the *empty* cells are represented as genes; original clues remain fixed. The main ideas:

- **Per-box initialization:** Each chromosome starts with every 3×3 box filled by its missing digits (shuffled), giving box-valid candidates from the beginning.
- **Box-preserving operators:**
 - *Crossover* (`custom_crossover()`): offspring inherit entire 3×3 boxes from parents.
 - *Mutation* (`custom_mutation()`): swaps two genes within the same box, preserving the box's multiset of digits.
- **Fitness** (`fitness_function()`): penalizes duplicates across rows, columns, and boxes. The GA maximizes the negative penalty; a valid Sudoku yields $\text{fitness} = 0$.

- **Solver** (`solve()`): configures and runs the GA (population, generations, selection, crossover/-mutation hooks), then decodes the best chromosome back into a grid.

This box-aware design constrains search to mostly fixing row/column conflicts, improving stability and time-to-solution.

2 Quick Start

1. Install dependencies: `pygad`, `numpy`.
2. Choose a puzzle from `SUDOKU_EXAMPLES`.
3. Run `__main__` and inspect the output grid and fitness.

3 Repository Structure

```
.
|-- session4/
|   |-- Sudoku.py
|   |-- sample/
|   |-- sudoku1.pdf
|   '-- sudoku2.pdf
```

4 Key Components

4.1 Class `SudokuSolver`

- `solve()`GA configuration and execution (returns best grid and fitness).
- `fitness_function()` ..Negative penalty based on duplicates in rows, columns, and boxes.
- `custom_crossover()`Box-wise inheritance from parents.
- `custom_mutation()`In-box gene swap operator.

4.2 Utilities

- `decode_solution`Map chromosome back to a 9×9 grid.
- `print_sudoku`, `analyze_solution`I/O and simple validity checks.

5 Example Snippet

```
1 solver = SudokuSolver(SUDOKU_EXAMPLES["expert"])
2 solution, fitness = solver.solve(num_generations=1000,
3 num_parents_mating=80,
4 sol_per_pop=400)
```